

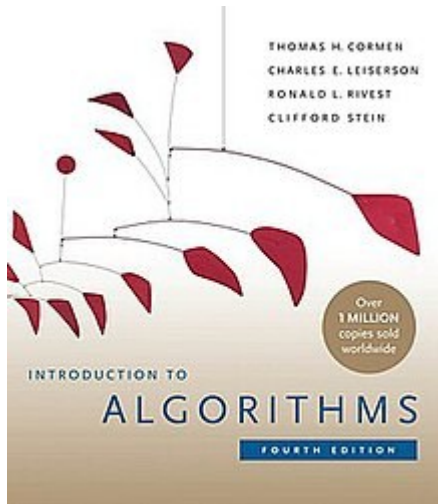
ADVANCED PROGRAMMING TECHNIQUES

LECTURE 1: INTRODUCTION

CONTENTS

- > Course overview
- > Why this course matters
- > A brief introduction to Linux
- > Fundamental concepts
- > Linux's programming interface

Course Overview



THE LINUX PROGRAMMING INTERFACE

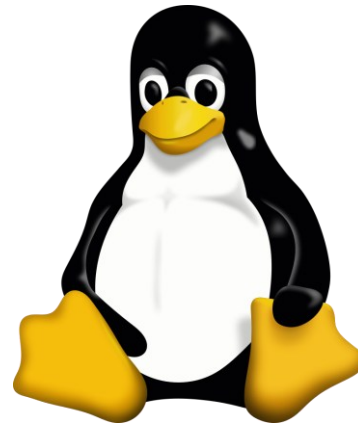
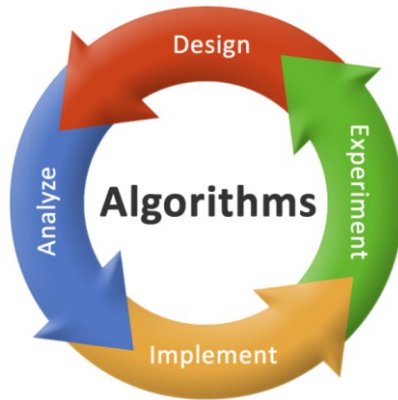
A Linux and UNIX System Programming Handbook

MICHAEL KERRISK



- > Lecturer: Hoang Gia Hung
 - R702, E3 building (by appointment).
- > Textbooks:
 - **CLRS**, "*Introduction to algorithms*", 4th Ed., The MIT Press, 2022.
 - Michael Kerrisk, "*The **L**inux **P**rogramming **I**nterface*", No Starch Press, 2010.
- > Software: Linux
 - C programming toolchain for Linux
- > Grading:
 - Labs - 20%
 - Midterm - 20%
 - Final exam - 60%

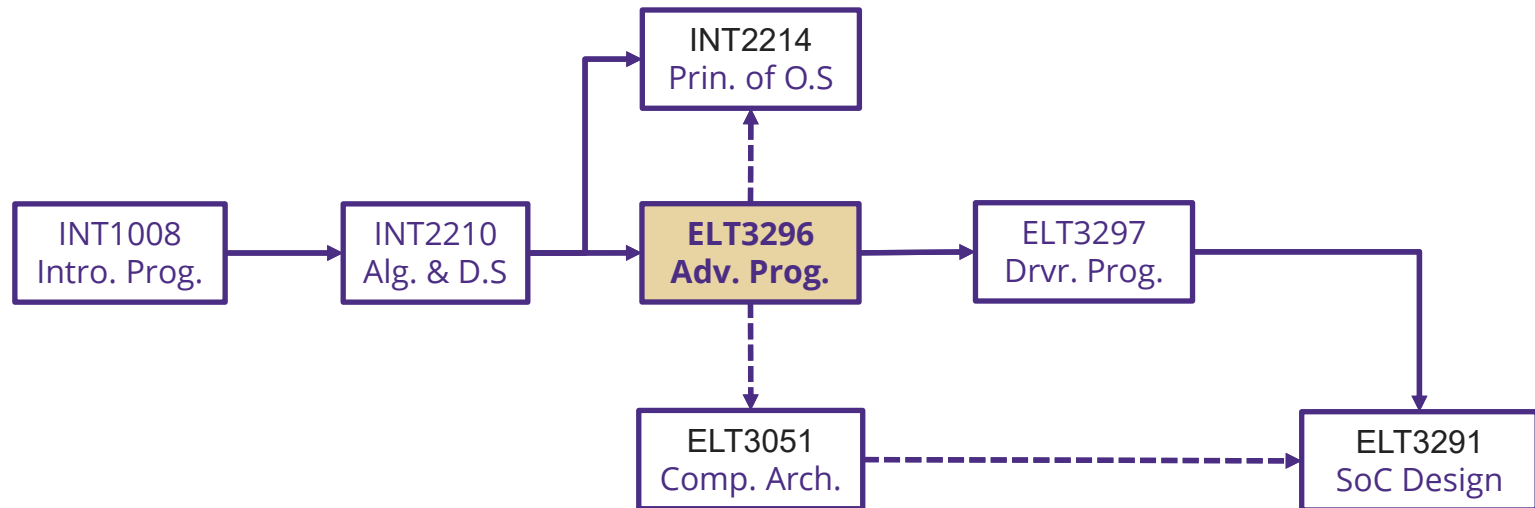
Course Objectives



1. Understand the organization of Linux Operating System
 - Linux Programming Interface
2. Able to analyze and design complex algorithms
 - Parallel computing, dynamic programming, optimization
3. Able to implement complex algorithms via Linux toolchains
 - Proficiency in shell scripting as well as C programming

Why This Course Matters (1)

> Because of its role in your program pathway

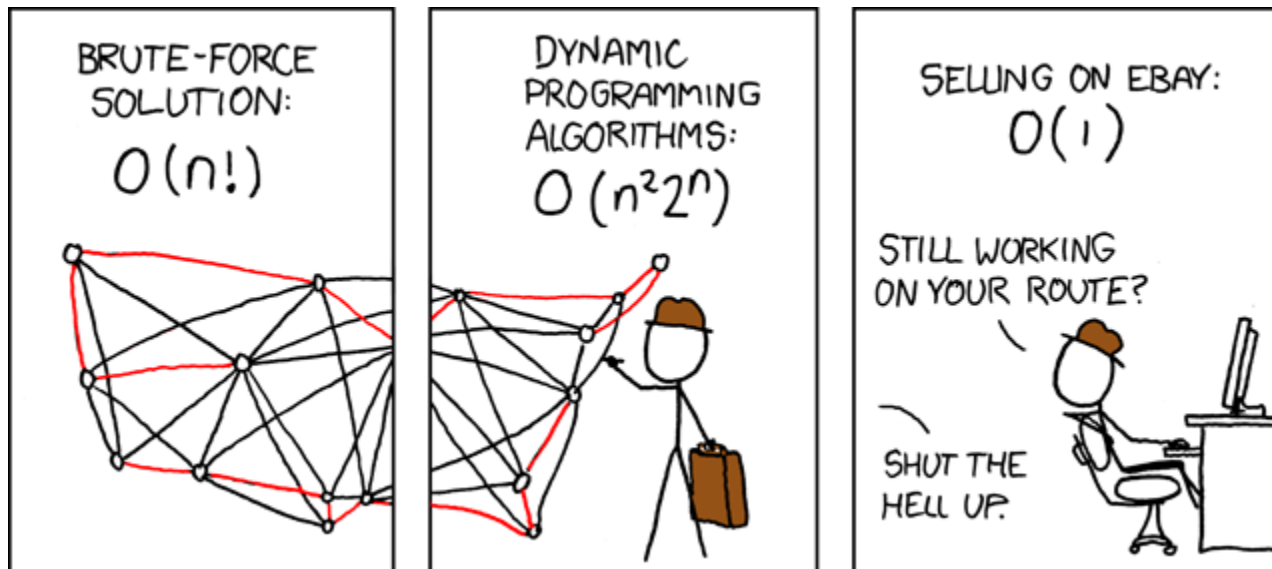


- > For its broad and far-reaching impact



Why This Course Matters (3)

- > It will improve your programming skills (system + application), making you a better **engineer** and **programmer**.



Why This Course Matters (4)

Posted 1 day ago **SUPER HOT**

Embedded Engineer (C++, Linux, >2 Yoe, Signing Bonus)

 MARUSYS VINA

💰 [Sign in to view salary](#)

 At office
 Ha Noi

Embedded C++ English

Posted 10 days ago **HOT**

IT Service Operation Manager (Cloud/ Linux/ English)

 SkyLab

💰 [Sign in to view salary](#)

 At office
 Ho Chi Minh

Cloud English Linux

Posted 2 days ago **HOT**

Embedded Software Engineer (C++, Linux)

 Datalogic Việt Nam

💰 [Sign in to view salary](#)

 At office
 Ho Chi Minh

C++ Linux Embedded

Posted 24 days ago

DevOps Engineer (Linux/ AWS) - Khối CNTT

 MB Bank

💰 [Sign in to view salary](#)

 At office
 Ha Noi

DevOps AWS Linux

Effective Learning Strategies

"Tell me and I forget; teach me and I may remember; involve me and I will learn"
Chinese proverb

- > Active learning
 - **Pre-class:** self-study the materials posted on Canvas at your own pace.
 - **In class:** we do not cover everything, slides are more like mental tabs to remind you of what was said in the book(s).
 - **After class:** do your homework and/or prepare for labs.
- > Don't feel rushed if you don't get something in class ...
 - ask questions at any time;
 - you might pick it up in labs.
- > When you receive a low grade ...
 - worry less about how it impacts your GPA;
 - worry more about the fact that you didn't learn as well as you should.

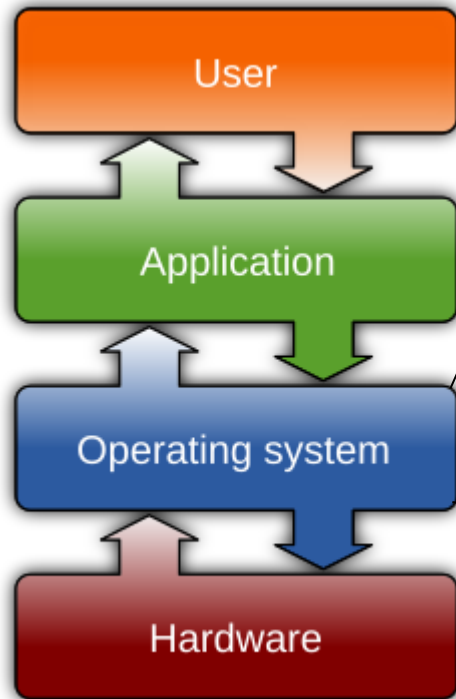
What is Linux?

- > A Unix-clone **operating system** written by Linus Torvalds
 - and a loosely-knit team of hackers across the Net
- > There're many **distributions** (Linux + GNU Utilities)



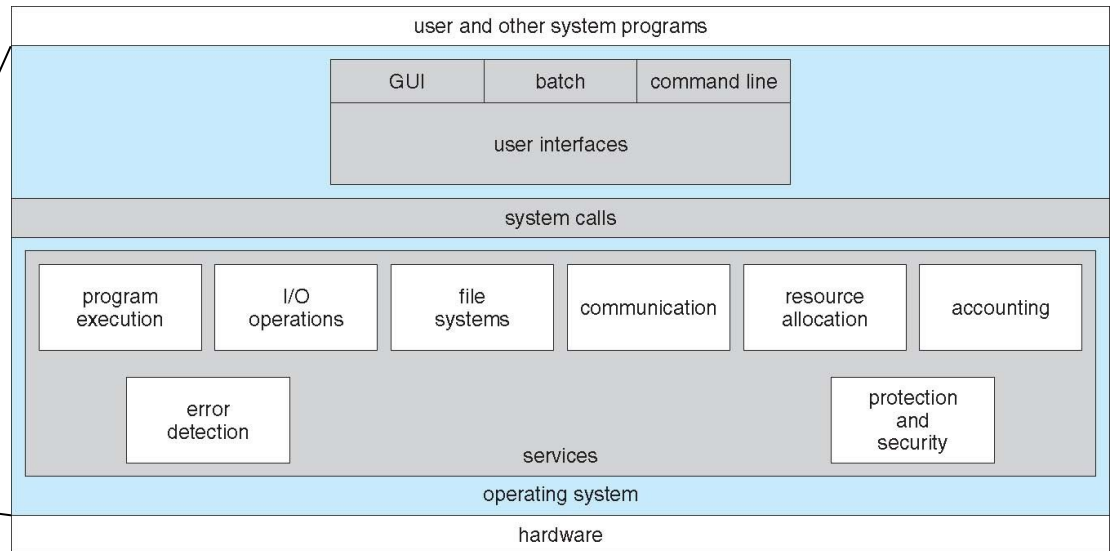
What is an Operating System?

User view



- > Intermediary SW for program execution

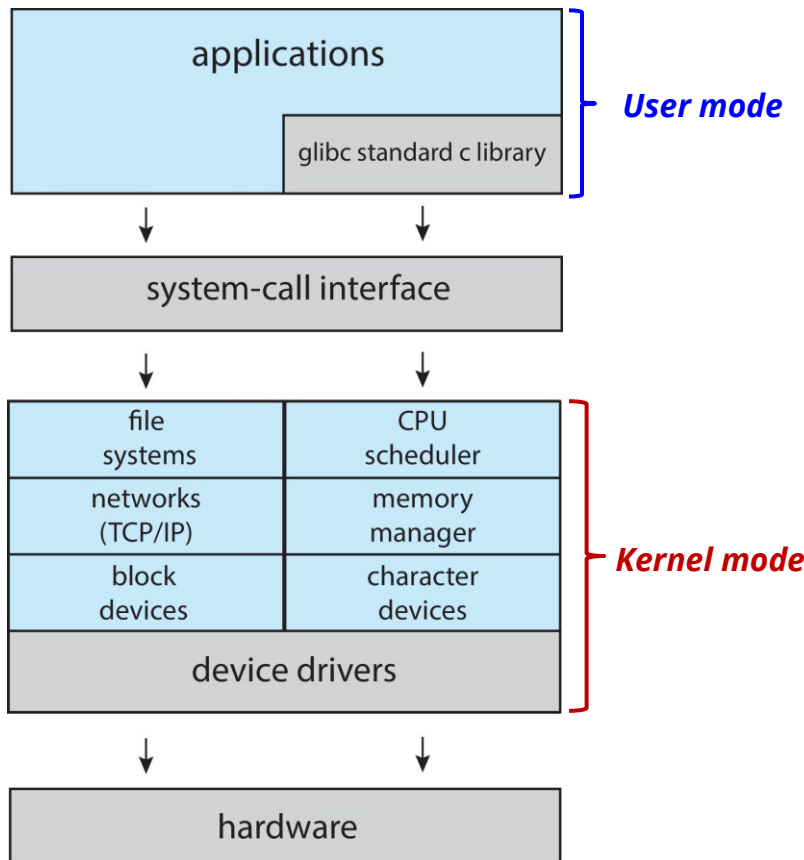
Functional view



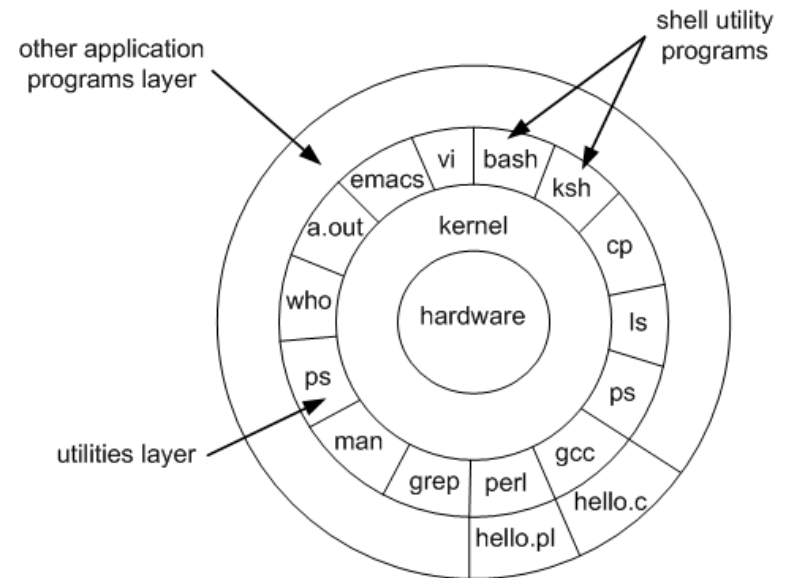
- > Provides services for user's interaction with HW
 - User interface (UI)
 - Program execution, I/O operations, file operations, etc.
- > Manages computer's resources efficiently
 - Resource allocation, accounting, protection & security

Structure of the Linux Operating System

Monolithic + modular design



Built on a layered architecture

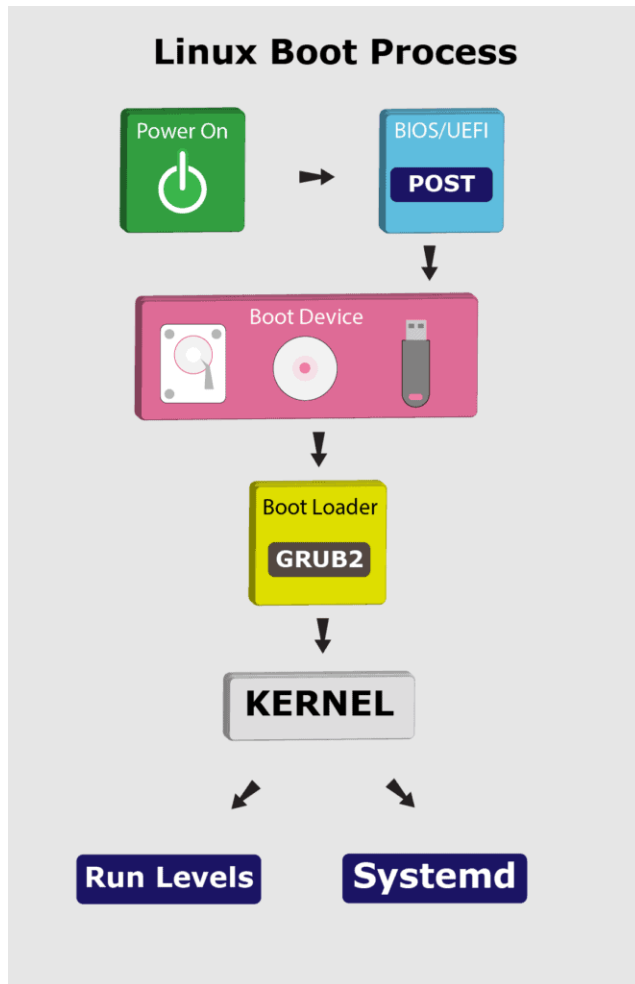


> Kernel

- Loaded at boot & running at all times
- Has privileged access to all resources

> System programs

How Linux Boots



> When the system is turned on

- Execution starts at a **fixed** memory location, running codes of the BIOS/UEFI
- BIOS/UEFI runs POST to ensure that the computer is in proper working condition.

> Now, the OS kernel must be initiated

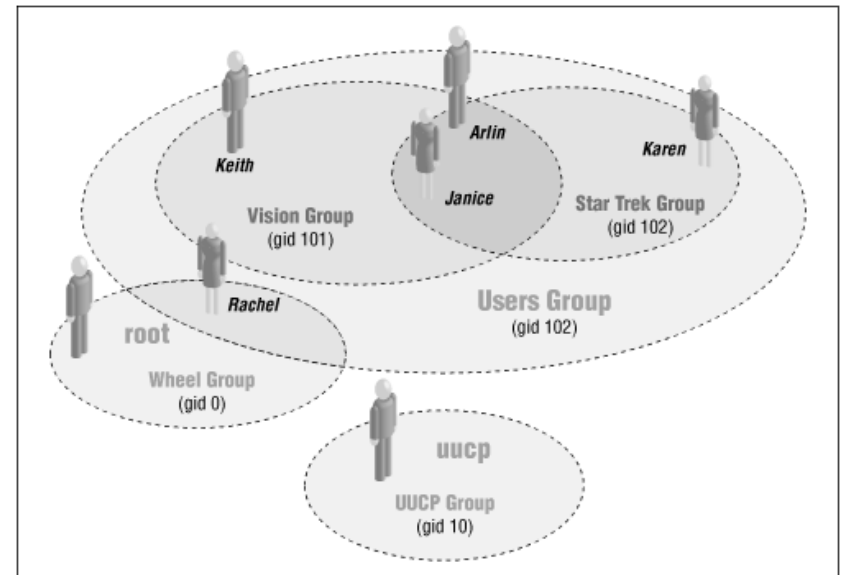
- BIOS/UEFI selects a boot device
- The **boot loader** on the boot device is executed
- The OS kernel is located, loaded, and executed.

> Once the OS gains control of the HW

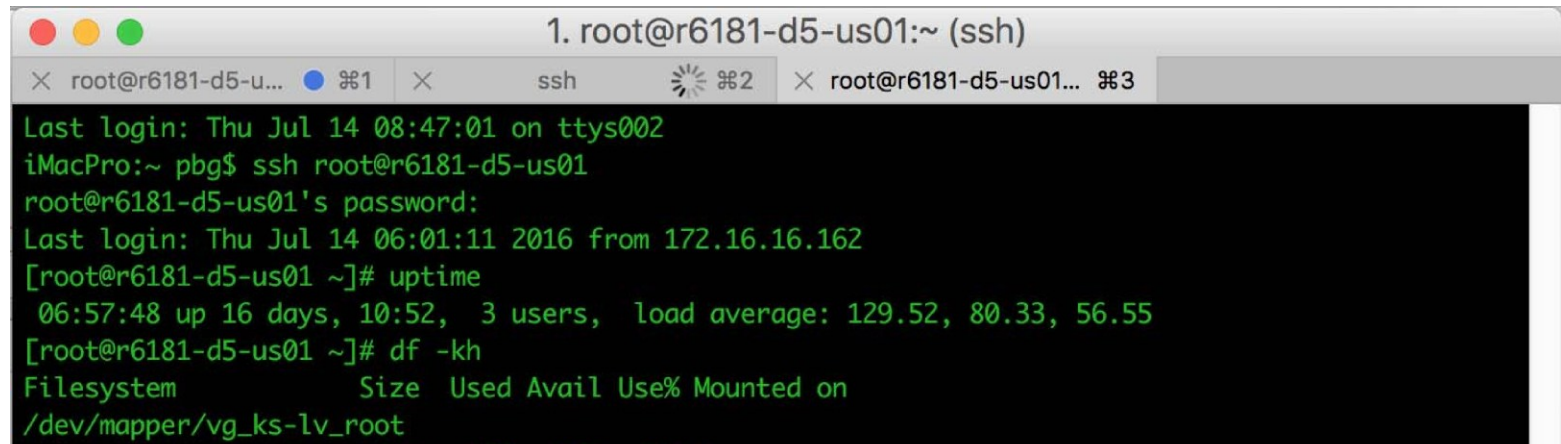
- The parent process (**systemd**) is started
- Systemd starts many other **daemons**, specified by the **run levels** the system boots into.
- Boot finished, system waits for events to occur.

Users and Groups in Linux

- > Every user of the system has a unique *login name* (username)
 - Inside Linux, each user is uniquely identified by a numeric *user ID* (UID)
- > Users are often organized into groups
 - Useful for controlling access to files and other system resources
 - Each group has a unique *group name* and *group ID* (GID)
- > Superuser
 - Has special privileges
 - UID = 0, login name = *root*



The Linux Shell



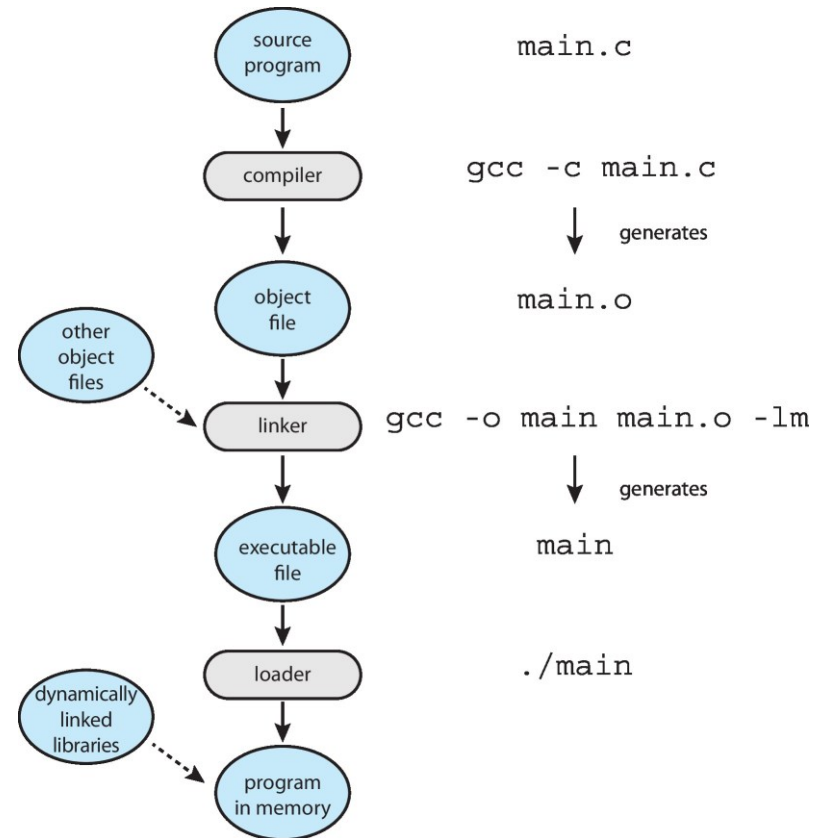
The screenshot shows a terminal window titled "1. root@r6181-d5-us01:~ (ssh)". The terminal content is as follows:

```
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
```

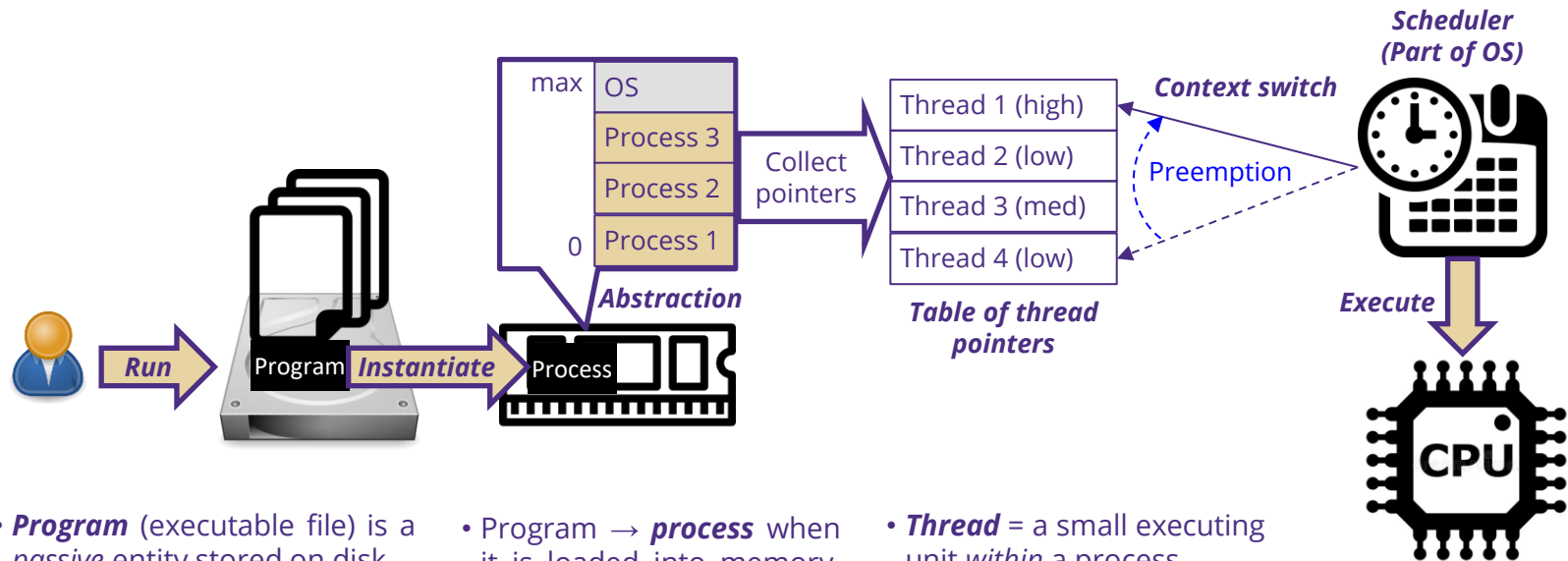
- > An interactive program acting as a *command interpreter*
 - Read user-typed commands & execute appropriate programs in response.
 - Is a user process: *login shell* is created to run a shell when the user logs in.
 - Bourne shell (sh), C shell (csh), Korn shell (ksh), Bourne again shell (bash).
- > Also capable of interpreting **shell script**
 - Text files containing shell commands, including loop and conditional statements, I/O commands, and functions.

Programs in Linux

- > Normally exist in 2 forms
 - Source code
 - Binary (executable)
 - Source code is converted to binary via compiling & linking
- > Apps compiled in Linux often not executable on other OS's
 - Unique system calls
 - Solutions:
 - ✓ Using a multi-OS *interpreted* language
 - ✓ Language includes a *virtual machine*
 - ✓ Compiled *separately* on each OS



Program Execution on Linux



- **Program** (executable file) is a *passive* entity stored on disk.
- Program execution started via GUI mouse clicks, CLI entry of its name, etc.
- Program → **process** when it is loaded into memory, process is *active*.
- Process execution must progress in *sequential*.
- **Multiprogramming**: CPU switches to another job in memory when the active job is waiting for I/O.
- **Multitasking**: CPU run multi processes simultaneously via time sharing.
- **Thread** = a small executing unit *within* a process.
- Resources *shared* among threads within a process.
- Possible to run multiple threads *concurrently*.
- A *scheduler* picks a thread if multiple ones are ready at the same time.

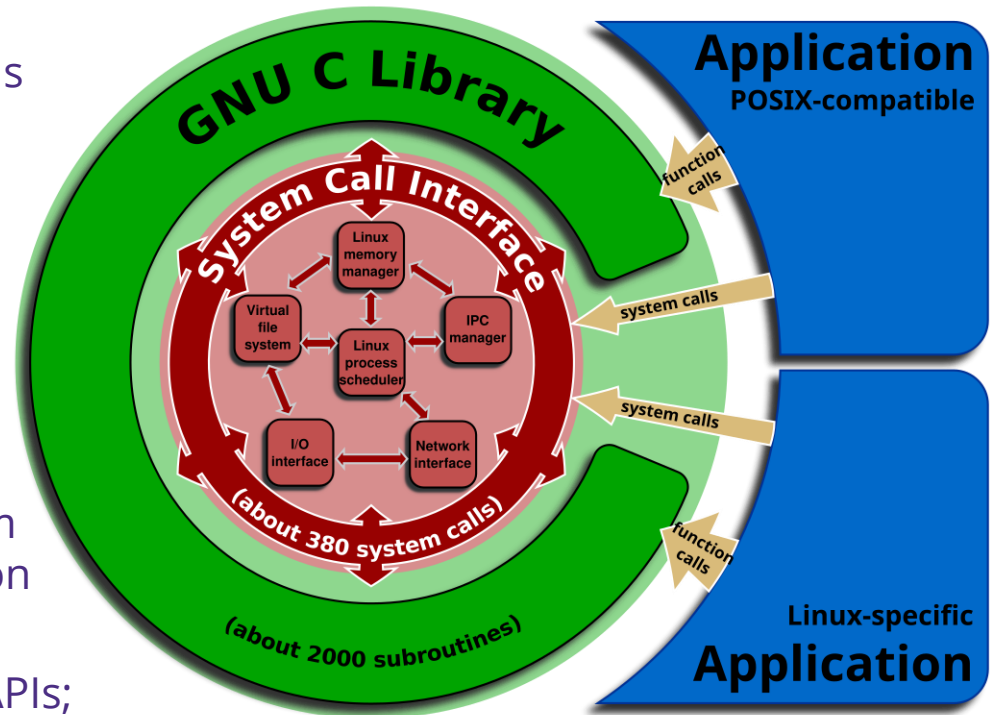
System Programming in Linux

> System Calls

- Interface between user programs and the Linux kernel.
- Explicit requests to kernel made via a **software interrupt** (trap).

> Standard C Library (glibc)

- Provides functions specifying how to obtain service (API).
- Wrapper routines around system calls, e.g `malloc` is a glibc function that use the `brk` system call.
- POSIX-compliant = a std. set of APIs; non-UNIX systems can be POSIX-compliant.



Common System Calls in Linux

> File Management

- `open()`, `read()`, `write()`, `close()`

> Process Management

- `fork()`, `exec()`, `wait()`

> Memory Management

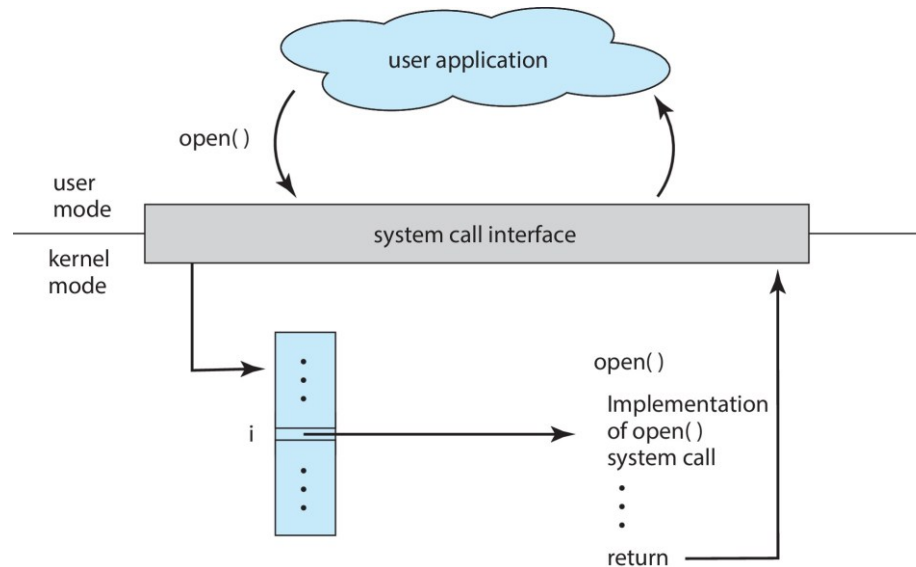
- `mmap()`, `brk()`

> Inter-Process Communication (IPC)

- `pipe()`, `shmget()`, `semop()`

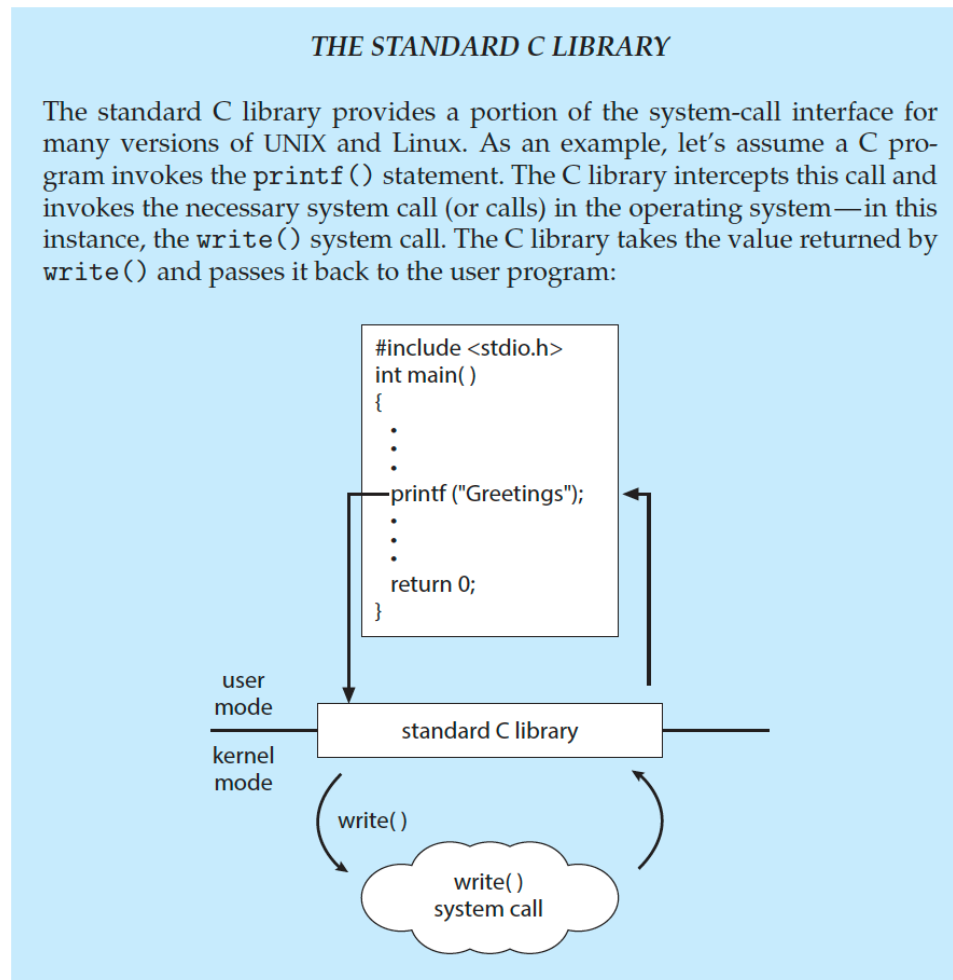
System Call Implementation

- > The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
 - Most details of OS interface hidden from programmer by API.
- > The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call.



Standard C Library Example

- > C program invoking printf() library call, which calls write() system call



Development Tools on Linux

- > Compilers
 - GCC, Clang.
- > Debuggers
 - GDB, Valgrind
- > Build Systems
 - Make, CMake
- > Version Control
 - Git
- > Integrated Development Environments (IDEs)
 - VS Code, Vim, Emacs

NEXT LECTURE

[Flipped class] Programming file I/O on Linux

- > Pre-class
 - Study pre-class materials on Canvas
- > In class
 - Reinforcement/enrichment discussion
 - Quizzes
- > Post class
 - Homework
 - Consultation (if needed)